AMRITA VISHWA VIDYAPEETHAM
DEEMED TO BE UNIVERSITY

Research

Amrita School of Computing
Amritapuri Campus

AMRITA VISHWA VIDYAPEETHAM
DEEMED TO BE UNIVERSITY UNDER SECTION 3 OF UGC ACT, 1956

SHARP-LLM

A Secure and Resource-Efficient Framework for Source Code Vulnerability Analysis and Healthcare-Specific Risk Prioritization

Basanth M S (AM.SC.R4CSE25007-PT), Gopakumar G

Dept. of Computer Science and Engineering, School of Computing
Amrita Vishwa Vidyapeetham, Amritapuri, India

01

Context of the Problem

Why This Matters for Healthcare Software

- Healthcare software handles some of the most sensitive data in any industry
  - Electronic Health Records (EHR) and patient data confidentiality
  - Clinical decision support and diagnostic workflows
  - Regulatory compliance (HIPAA, GDPR, DISHA, DPDPA)
- A vulnerability in healthcare software is not just a bug — it can compromise:
  - Patient safety and data privacy
  - Service continuity of critical systems
  - Regulatory compliance and institutional trust
- Existing approaches have key limitations:
  - Traditional SAST tools: high false-positive rates, limited generalization
  - LLM-based methods: computationally expensive, no domain-specific guidance
  - No framework bridges code analysis → healthcare-specific risk prioritization

02

Research Problem

Two Key Research Gaps

- Gap 1: No Domain-Specific Risk Prioritization
  - Literature focuses on detection accuracy alone — all CWEs treated equally
  - A buffer overflow in an EHR system ≠ a buffer overflow in a game engine
  - No framework translates detection results into healthcare-relevant risk signals

- Gap 2: Limited Healthcare-Oriented Security Resources
  - Privacy-aware vulnerability resources for healthcare are scarce
  - Missing link: mapping technical CWE findings to patient-data and compliance risks

03

Main Contributions

Contributions

- Contribution 1: Healthcare-Oriented Risk Prioritization Pipeline
  - Three-stage post-analysis pipeline: CWE → LINDDUN threat types → Policy control domains → Weighted risk score
  - Produces ordinal priority levels (Critical / High / Medium / Low) for remediation
  - Transforms raw CWE predictions into actionable, healthcare-relevant risk signals

- Contribution 2: Lightweight, Practical CWE Classification and Analysis
  - Fine-tuned models (35M–125M params) run on consumer-grade GPU
  - Quick inference — viable for CI/CD, IDE plugins, real-time code review
  - 118 CWE categories classified (multi-class, not just binary detection)

04

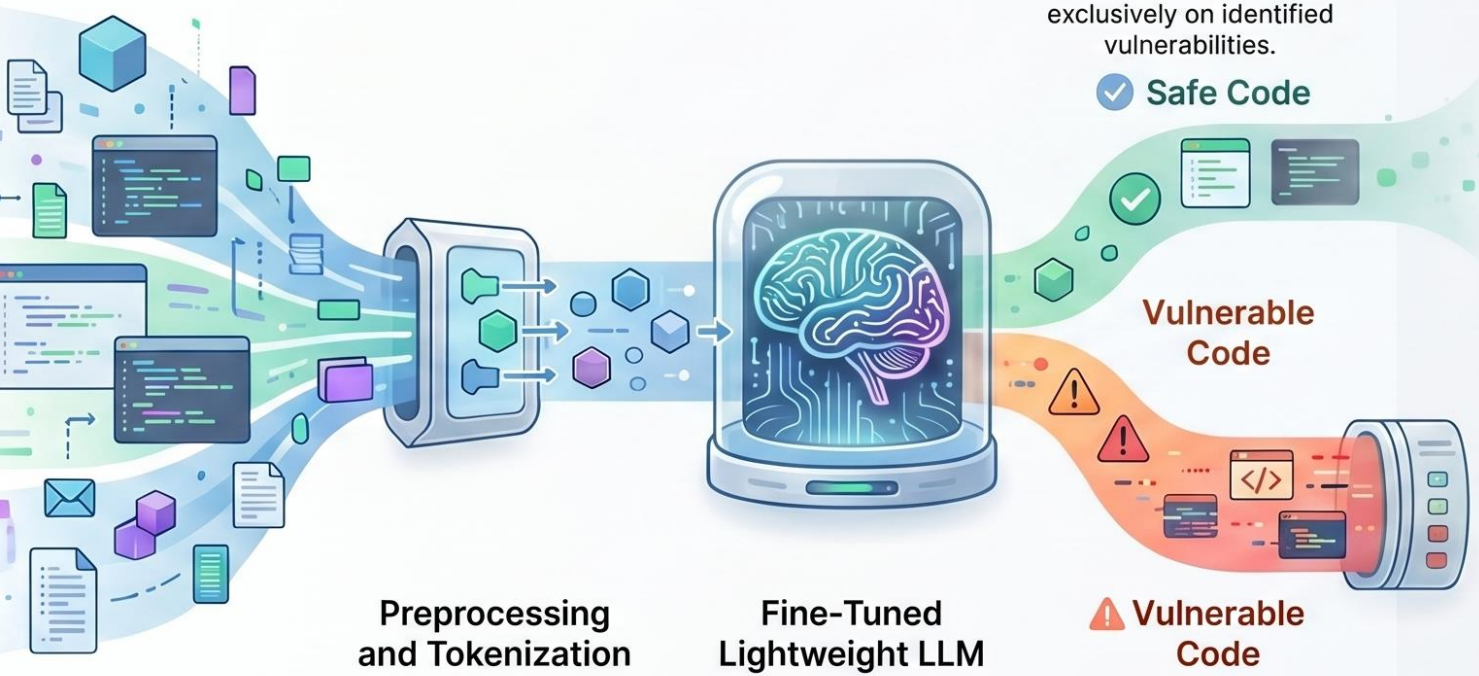
Methodology

SHARP-LLM: Healthcare-Oriented Code Risk Pipeline

Automated Code Analysis

Binary Code Classification

The system filters out safe code to focus exclusively on identified vulnerabilities.



Raw Source Code

Preprocessing and Tokenization

Raw source code is prepared and broken down for machine analysis.

Fine-Tuned Lightweight LLM

A specialized model evaluates the code to distinguish safe from vulnerable segments.

Risk Mapping and Prioritization

CWE Association and Categorization

Vulnerabilities are linked to Common Weakness Enumerations and security risk categories.

Policy-Relevant Control Domains

Threats are mapped to specific regulatory and organizational security control domains.

Healthcare-Oriented Risk Output

Generates a prioritized report focused on clinical and healthcare security impacts.

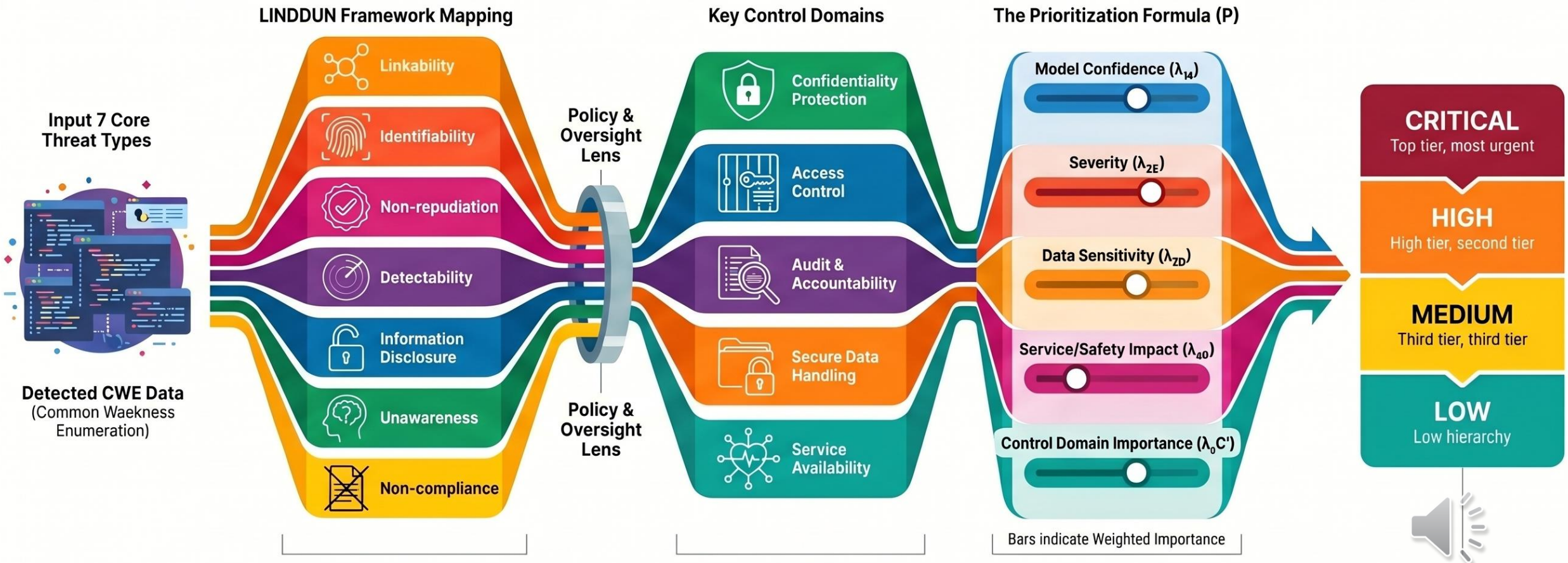
A Framework for Healthcare Risk Prioritization

STEP 1 — **Privacy & Security Risk Categorization**

STEP 2 — **Policy-Relevant Control Domains**

STEP 3 — **Weighted Risk Prioritization**

FINAL OUTPUT **& IMPACT**



Risk Categorization Formula
R = f(c) (Risk is a function of identified threat categories)

Control Mapping Formula
C = g(R) (Controls derived based on risk profile)

Five Critical Weighting Factors
The formula evaluates five critical weighting weighting factor. Impact, Control domain importance

Strategic Remediation Decisions:
Security actions aligned with patient-data confidentiality, service continuity, and regulatory compliance.

05

Main Results

Model Comparison — CWE Classification Performance

- Fine-tuned Encoder Models (full test set, 22,447 samples):
  - CodeT5-Base: Macro-F1 = 0.9605, Accuracy = 99.73%, Latency = 24.9 ms
  - CodeT5-Small: Macro-F1 = 0.9533, Accuracy = 99.65%, Latency = 17.2 ms
  - GraphCodeBERT: Macro-F1 = 0.9530, Accuracy = 99.64%, Latency = 26.9 ms
  - CodeBERT: Macro-F1 = 0.9512, Accuracy = 99.60%, Latency = 19.4 ms

- QLoRA Fine-tuning:
  - CodeGemma-2B: Macro-F1 = 0.9168, only 0.5% params trained, ~1.8 GB memory

Key Findings

- Lightweight fine-tuning outperforms large zero-shot models
  CodeT5-Base (110M) performed better than Gemma-4 (5.13B)

- Architecture matters more than parameter count
  CodeT5-Base (110M) > GraphCodeBERT (125M) > CodeBERT (125M)

- Consumer-grade GPU is sufficient for production-quality classification
  All encoder models train on 8 GB VRAM, with good inference response.

- Risk prioritization layer adds clinical context that raw CWE labels lack
  Same CWE → different priority depending on healthcare data exposure

06

Critical Review

What the Paper Does Not Do — And Can Improve

- Synthetic dataset only (Juliet Test Suite)
  - Template-generated code lacks real-world complexity (multi-file, third-party libraries)
  - Models may not generalize to production healthcare codebases without further validation

- Risk prioritization layer is not yet empirically validated
  - The CWE → LINDDUN → priority mapping is proposed but not tested on real healthcare incidents
  - Future work: apply pipeline to real CVE databases from healthcare breaches

- Sequence length constraint (256 tokens)
  - Sufficient for Juliet's short functions; may cause information loss on real-world code

Thank You